⚡ Quick Start Guide — Running TelcoBot

Get the TelcoBot chatbot running in under 5 minutes.

Prerequisites

Requirement	Version	Check Command
Python	3.10+	`python --version`
pip	Latest	`pip --version`
OpenAI API Key	—	Get one here (https://platform.openai.com/api-keys)

Step 1: Install Dependencies

```
# Navigate to the project directory
cd telecom_support_chatbot

# (Recommended) Create and activate a virtual environment
python -m venv venv
source venv/bin/activate          # macOS / Linux
# venv\Scripts\activate           # Windows

# Install all required packages
pip install -r requirements.txt
```

What gets installed:

Package	Purpose
`langchain`, `langchain-core`	LLM orchestration framework
`langchain-openai`	OpenAI LLM & embedding integration
`langgraph`	Graph-based agent workflow
`langchain-text-splitters`	Document chunking for RAG
`chromadb`, `langchain-chroma`	Vector database for knowledge base
`gradio`	Web-based chat interface
`python-dotenv`	Environment variable loading
`langsmith`	Tracing & observability (optional)

Step 2: Set Up Your OpenAI API Key

Option A — Environment file (recommended)

```
# Copy the example .env file
cp .env.example .env

# Edit the .env file and paste your API key
nano .env   # or use any text editor
```

Your `.env` file should look like:

```
OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxx
```

Option B — Export directly in terminal

```
export OPENAI_API_KEY="sk-proj-xxxxxxxxxxxxxxxxxxxx"
```

Option C — Already configured globally

If your `OPENAI_API_KEY` is already set in your system environment variables (e.g., in `~/.bashrc` or `~/.zshrc`), you can skip this step — the app will pick it up automatically.

> 💡 **Tip:** You can verify your key is set by running:
> `bash`
> `echo $OPENAI_API_KEY`

Step 3: Run the Application

```
python app.py
```

What happens when you run it:

```
🚀 Initializing TelcoBot …
  📦 Setting up SQLite database …          ← Creates/seeds the customer database
  🧠 Setting up RAG knowledge base …       ← Builds vector store from knowledge docs
  ↳ 3 documents → 24 chunks                ← Chunks & embeds knowledge base
  ✅ RAG knowledge base ready.
  🔧 Building LangGraph workflow …          ← Compiles the agent graph
  ✅ Workflow compiled.
✅ TelcoBot is ready!

Running on local URL:  http://localhost:7860    ← Open this URL!
```

Step 4: Access the Chatbot

Open your browser and navigate to:

🌐 http://localhost:7860

You'll see the TelcoBot interface with:
- A chat window
- A text input box
- Example prompts to try
- A "Clear Chat" button

Step 5: Try It Out!

Here are some example prompts to test different agents:

💰 Billing

```
What's the latest bill for customer CUST-1001?
Does CUST-1002 have any outstanding balance?
Show me payment history for CUST-1004
```

📋 Plans

```
What plans do you offer?
Compare the Standard and Premium plans
Tell me about the Family plan
```

🔧 Technical Support (uses RAG)

```
My data speed is very slow, what should I do?
How do I set up voicemail?
Can I use my phone internationally?
```

📈 Usage Tracking

```
Show me the usage summary for CUST-1001
What's the daily usage breakdown for CUST-1003?
```

👤 Account Management

```
Look up a customer named Alice
Show me the profile for CUST-1006
Create a support ticket for CUST-1002 about slow internet
```

💬 General

```
Hi, what can you help me with?
```

🧠 Multi-turn Memory Test

Try this sequence to test conversation memory:

```
1. "What's the latest bill for CUST-1001?"
2. "What plan are they on?"              ← Remembers CUST-1001 from turn 1
3. "Any open tickets for them?"          ← Still remembers the customer
```

Available Sample Customer IDs

Customer ID	Name	Plan	Status
`CUST-1001`	Alice Johnson	Standard ($49.99/mo)	Active
`CUST-1002`	Bob Martinez	Premium ($79.99/mo)	Active
`CUST-1003`	Carol Chen	Basic ($29.99/mo)	Active
`CUST-1004`	David Williams	Unlimited ($99.99/mo)	Active
`CUST-1005`	Eva Brown	Prepaid Lite ($15.00/mo)	Suspended
`CUST-1006`	Frank Davis	Family ($119.99/mo)	Active

Optional: Enable LangSmith Tracing

For debugging and observability, add these to your `.env`:

```
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=ls-your-langsmith-api-key
LANGCHAIN_PROJECT=telecom-support-chatbot
```

Then view traces at https://smith.langchain.com (https://smith.langchain.com).

Troubleshooting

"ModuleNotFoundError: No module named 'X'"

```
pip install -r requirements.txt
```

"Error code: 401 — Incorrect API key"

- Verify your `.env` file has the correct key
- Make sure there are no extra spaces or quotes around the key
- Try: `echo $OPENAI_API_KEY` to verify it's set

"Address already in use" (port 7860)

```
# Option 1: Kill the existing process
lsof -ti :7860 | xargs kill -9

# Option 2: Use a different port
python -c "from app import create_interface; cre-
ate_interface().launch(server_port=7861)"
```

RAG knowledge base not loading

```
# Delete the cached vector store and restart
rm -rf data/chroma_db/
python app.py
```

Slow first startup

The first run builds the ChromaDB vector store (embeds all knowledge-base documents). Subsequent runs load from cache and are much faster.

Stopping the Application

Press `Ctrl + C` in the terminal to stop the server.

Project Structure Reference

```
telecom_support_chatbot/
├── app.py                    ← Main entry point (run this!)
├── requirements.txt          ← Dependencies
├── .env                      ← Your API keys (create from .env.example)
├── agents/graph.py           ← LangGraph workflow
├── tools/                    ← 13 tools across 5 files
├── data/database.py          ← SQLite setup & seed data
├── knowledge_base/           ← RAG source documents (3 markdown files)
├── config/settings.py        ← All configuration settings
├── screenshots/              ← Application screenshots
├── README.md                 ← Full documentation
└── workflow_diagram.md       ← Mermaid workflow diagrams
```